

@pipeline-graph

Appears in: **DL-02**

pipeline/graph.py:13-31

"The Pipeline came first — six nodes, ingest then detect then evaluate then decide then act then audit, with one typed state object passed through each node (@pipeline-graph)."

@pipeline-graph

pipeline/graph.py:13-31

```
13  def build() -> "CompiledGraph":
14      g = StateGraph(PipelineState)
15
16      g.add_node("ingest", ingest.node)
17      g.add_node("detect", detect.node)
18      g.add_node("evaluate", evaluate.node)
19      g.add_node("decide", decide.node)
20      g.add_node("act", act.node)
21      g.add_node("audit", audit.node)
22
23      g.set_entry_point("ingest")
24      g.add_edge("ingest", "detect")
25      g.add_edge("detect", "evaluate")
26      g.add_edge("evaluate", "decide")
27      g.add_edge("decide", "act")
28      g.add_edge("act", "audit")
29      g.add_edge("audit", END)
30
31      return g.compile()
```

What it shows: The LangGraph `build()` that wires the six nodes in a fixed sequence (ingest → detect → evaluate → decide → act → audit → END), all sharing one typed `PipelineState`.

Source: pipeline/graph.py:13-31 · image rendered by build_snippets.py